Qwen3 Loop-Trigger Attention Query Comparison

2026-04-14

Object

This note compares three query positions on the same saved March 22–23 Qwen3-1.7B instruct prompt-profile rows:

- `trigger_end`: the final token of the twentieth repeated 30-gram
- `first_trigger_token`: the first token of that same twentieth repeated 30-gram
- `trigger_start`: the token immediately before that twentieth repeated copy begins

The naming change is deliberate. The earlier 2026-04-14 note had called the first-token object `trigger_start`. I am no longer using that name, because Wangzhi's intended object was the token right before the final repeated copy, not the first token inside it.

Everything else is held fixed:

- model: `Qwen/Qwen3-1.7B`
- loop definition: `n=30`, `k=20`
- selection rule: recomputed trigger prefix must exactly match saved `first_loop_prefix_length`
- selected rows: `811 / 820` replayable loop rows
- dataset counts: `GPQA 121`, `AIME 30`, `MATH-500 68`, `MMLU-Pro 130`, `LiveCodeBench 462`
- total prefix lengths: median `9846`, `p95 = 22321`, max `28830`

There is still no extra mid-prefix truncation inside any run. Each forward pass sees the full prompt plus the completion prefix through the saved first loop trigger.

Reconstruction Boundary

The March prompt-profile archives preserve exact `prompt_token_ids`, but they do not preserve exact `completion_token_ids`. For those older rows, the replay path still retokenizes saved `completion_text`.

Dataset	Rollouts	Loop rows	Exact length	+1 hidden stop	Trigger-prefix match
GPQA	792	124	46 / 792	749 / 792	121 / 124
AIME	240	31	38 / 240	234 / 240	30 / 31
MATH-500	2000	68	38 / 2000	1991 / 2000	68 / 68
MMLU-Pro	3200	130	34 / 3200	3191 / 3200	130 / 130
LiveCodeBench	3200	467	356 / 3200	3168 / 3200	462 / 467
Overall	9432	820	512 / 9432	9333 / 9432	811 / 820

So the replay boundary is still: exact prompt IDs, approximate completion IDs on these old archives, and exact trigger-prefix recovery on `811 / 820` loop rows.

Measurement Definition

For each selected row:

1. Recompute the loop trigger with `find_ngram_loop_trigger(..., n=30, k=20)`.
2. Keep only rows whose recomputed `first_loop_prefix` matches the saved `first_loop_prefix_length`.
3. Replay the model on `prompt_token_ids + completion_token_ids[:first_loop_prefix_length]`.
4. Apply an explicit causal mask inside the probe wrapper so keys with `key_position > query_position` are forced to zero.

That explicit mask matters. The earlier first-token note had been leaking attention onto future trigger tokens. The first-token and pre-trigger numbers below replace that older surface.

The three query positions are:

- `trigger_end = prompt_len + trigger_end`
- `first_trigger_token = prompt_len + trigger_start`
- `trigger_start = prompt_len + trigger_start - 1`

Key positions are partitioned into disjoint bins:

- `prompt`
- `previous_loop`
- `current_trigger`
- `recent_nonloop`
- `other_completion`

`last_previous_loop` is still tracked as a subset of `previous_loop`, but it is not plotted as a separate mass line here.

There is no separate self bin. The query token contributes to the region it belongs to. So `current_trigger` at `first_trigger_token` includes the first trigger token/self, while `current_trigger` at corrected `trigger_start` is exactly zero because the full trigger span is in the future.

Summary statistics are:

- `mean_*_mass`: summed attention mass on that bin, averaged across heads, then across selected rows
- `top1_fraction_*`: fraction of heads whose argmax lands in that bin

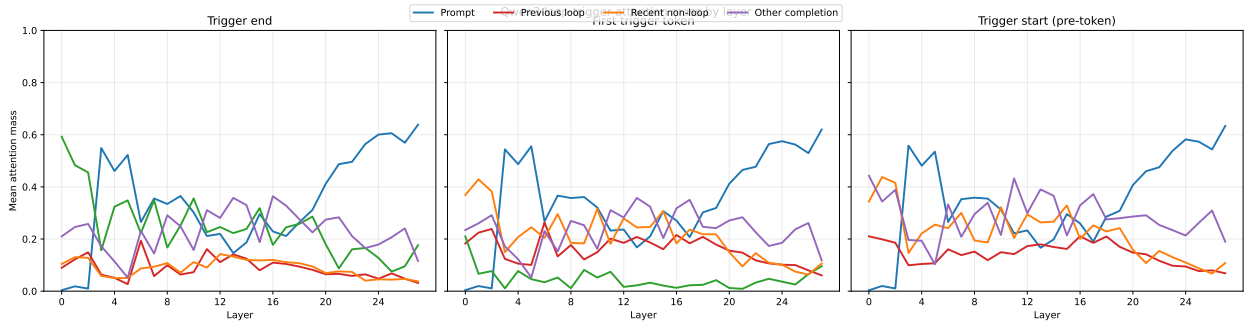

Figure 1: Row-weighted overall attention-mass progression from layer 0 to layer 27 for the three query positions.

Layer-by-Layer Progression

`attention_mass_by_layer_comparison.pdf` plots row-weighted overall bin mass from layer 0 to layer 27.

Main read from the curves:

- `trigger_end`: previous-loop mass rises early, peaks at layer 6 (`0.193`), and then falls as the final layer becomes prompt-dominant.
- `first_trigger_token`: previous-loop mass strengthens further and still peaks at layer 6 (`0.263`), while `current_trigger` shrinks to self-only and is only `0.034` at that peak.
- corrected `trigger_start`: `current_trigger` is identically `0`, previous-loop mass peaks later at layer 16 (`0.211`), and the strongest competition there is between `prompt` (`0.260`) and `other_completion` (`0.329`).

So the off-by-one correction does not erase previous-loop attention. It removes the spurious future-trigger mass and changes the curve shape: the clean pre-trigger object has a later, broader previous-loop peak than either of the two inside-the-trigger queries.

Final-Layer Overall Comparison

The final-layer summary is layer 27 for all three objects.

Query	Prev-loop	Prompt	Current-trigger	Recent-nonloop	Other-completion	Top-1 prev	Top-1 prompt	Top
`trigger_end`	0.031	0.639	0.176	0.038	0.116	0.0003	0.874	
`first_trigger_token`	0.061	0.620	0.096	0.106	0.118	0.019	0.831	
corrected `trigger_start`	0.069	0.634	0.000	0.108	0.190	0.027	0.872	

This comparison says three separate things:

- moving from `trigger_end` to `first_trigger_token` roughly doubles final-layer previous-loop mass (`0.031 -> 0.061`) and cuts current-trigger mass nearly in half (`0.176 -> 0.096`) once future-token leakage is removed;

- moving one token earlier again to corrected `trigger_start` removes `current_trigger` entirely, nudges previous-loop mass slightly higher (`0.069`), and shifts that mass into `other_completion` rather than back into the trigger span;
- prompt tokens remain the dominant late-layer target in all three views.

Corrected Trigger-Start By Dataset

For the corrected `trigger_start` object, `current_trigger` is zero on every dataset by construction, so the dataset-level table only reports the remaining bins.

Dataset	Rows	Prev-loop	Prompt	Recent-nonloop	Other-completion	Top-1 prev	Top-1 prompt
GPQA	121	0.045	0.648	0.122	0.184	0.011	0.872
AIME	30	0.107	0.628	0.082	0.184	0.050	0.869
MATH-500	68	0.082	0.619	0.121	0.178	0.035	0.837
MMLU-Pro	130	0.032	0.643	0.130	0.196	0.005	0.909
LiveCodeBench	462	0.081	0.630	0.098	0.191	0.035	0.868
Overall row-weighted	811	0.069	0.634	0.108	0.190	0.027	0.872

Dataset-level scientific read:

- `AIME` shows the strongest final-layer previous-loop mass (`0.107`).
- `MATH-500` and `LiveCodeBench` are next strongest at about `0.08`.
- `MMLU-Pro` is the weakest on this object (`0.032` prev-loop mass, `0.005` top-1 prev-loop).
- Every dataset still stays prompt-dominant in the final layer: prompt mass is `0.619-0.648`, and top-1 prompt is `0.837-0.909`.

Conclusion

The corrected read is now:

- the earlier first-token report needed a causal-mask fix and should be treated as superseded;
- the clean pre-trigger object does show nontrivial attention to previous loops:
  - final-layer previous-loop mass `0.069`
  - final-layer top-1 previous-loop `0.027`
  - overall previous-loop peak at layer 16 with mass `0.211`
- but the final layer is still mostly prompt-focused rather than previous-loop-focused:
  - final-layer prompt mass `0.634`
  - final-layer top-1 prompt `0.872`

So the strongest honest statement is not "Qwen3 is not paying attention to previous loops." The stronger-supported statement is:

> At the token immediately before the twentieth repeated copy begins, the final layer is still mostly prompt-focused, while previous-loop evidence is present and peaks in the middle of the stack.

Artifact Bundle

Updated raw/report artifacts now live under:

- `outputs/qwen3_loop_trigger_attention_full_20260414_rerun/`
- `outputs/qwen3_loop_trigger_attention_trigger_start_20260414/` (corrected first-token bundle)
- `outputs/qwen3_loop_trigger_attention_pre_trigger_start_20260414/` (corrected trigger-start bundle)